SHOPX

E-commerce Order Fulfillment & Delivery Analytics

SAP ERP → Python ETL → MySQL Data Warehouse → Power BI

Prepared by:

Harshita Fogat

harsfogat.ext@deloitte.com

---

1. Objectives

This section defines the business context, the problems being solved, and the KPIs used to measure success which will establish analytical intent before any technical work begins.

1.1 Problem Statement

ShopX operates its sales, inventory, and logistics on SAP ERP. Transactional data lives across multiple SAP modules - sales orders, deliveries, shipments, and customer records — but no single analytical layer exists to connect them. Business teams cannot answer basic questions about how quickly orders are processed, which carriers are underperforming, or how much revenue is at risk in open orders.

The core problems this case study addresses are:

- **No visibility into order fulfillment speed:** There is no measurement of time between order placement and shipment dispatch. Management cannot identify bottlenecks in the order processing pipeline.

- **No delivery performance tracking:** On-time delivery rate and carrier-level performance are unknown. The business cannot hold carriers accountable or benchmark improvement.
- **Fragmented data across SAP modules:** Customer, order, delivery, and shipment data sit in separate SAP tables (VBAK, VTTK, LIKP, KNA1) with no pre-built integration layer for analytics.
- **No revenue risk visibility:** The proportion of revenue tied to open or undelivered orders — the backlog — is not quantified, creating hidden commercial exposure.
- **No customer satisfaction linkage:** CSAT data does not exist in SAP, leaving the business unable to connect delivery outcomes to customer experience.

1.2 Business Goal

Build a complete, production-approach analytics system that transforms raw SAP transactional data into a trusted, queryable data warehouse and delivers an interactive Power BI dashboard. The system should enable ShopX to:

1. Monitor and reduce order processing time across all customer segments.
2. Track on-time delivery performance at carrier level and identify underperformance.
3. Quantify revenue backlog and open order exposure for commercial risk management.
4. Analyse customer-level revenue concentration and satisfaction trends.
5. Support data-driven decisions on carrier selection, fulfillment operations, and customer prioritisation.

1.3 KPIs - What We Are Measuring and Why

The following KPIs were defined upfront, before any ETL work began, to ensure the data model and warehouse design were aligned with business outcomes:

KPI	DAX Measure	Value
Order Processing Time	`Avg Processing Days`	**7.2 days avg**
On-Time Delivery Rate	`On-Time Rate %`	**50.0%**
Average Delivery Duration	`Avg Delivery Days`	**0 days (data limitation — see Note)**
Carrier Performance	`Carrier On-Time Rate %`	**A: 38% | B: 67% | C: 50%**
Total Revenue	`Total Revenue`	**$229,000**

Open Revenue (Backlog)	`Open Revenue`	**$118,000**
Order Fulfillment Rate	`Avg Fulfillment Rate %`	**100%**
Customer Satisfaction	`Avg CSAT Score`	**~4.2 / 5**

> **NOTE:** Average Delivery Duration shows 0 days because shipment_date and delivery_date are identical in the source SAP dataset. This is a flat-file export limitation. The on-time delivery metric was re-engineered to use shipment_status = 'Delivered' as the business rule instead of a duration threshold.

2. ETL Pipeline

The full pipeline was built in Python using pandas and SQLAlchemy, executed inside a Jupyter Notebook. No external ETL frameworks were used - every step was coded explicitly for full visibility and control.

2.1 Source Data & Extraction

The SAP dataset was provided as SAP-DataSet.xlsx — an eight-sheet Excel workbook simulating a real SAP flat-file export. Each sheet corresponds to a SAP module output:

Field	Description
`KNA1`	Customer Master — customer_id, customer_name, country, region, city
`LFA1`	Vendor/Carrier Master — vendor_number (→ carrier_id), vendor_name
`VBAK`	Sales Order Header — sales_document (→ order_id), order_date, customer_id, order_status
`VBAP`	Sales Order Items — item_number, material_number, quantity, net_price

LIKP	Delivery Header — delivery_number, delivery_date, sales_document, delivery_priority
LIPS	Delivery Items — delivered_quantity, net_price, batch_number, plant
VTTK	Shipment Header — shipment_number (→ shipment_id), shipment_date, carrier, shipment_status
VTTP	Shipment Items — shipped_quantity, material_number

```
Sheets found: ['KNA1', 'LFA1', 'VBAK', 'VBAP', 'LIKP', 'LIPS', 'VTTK', 'VTTP']

=== Row counts after extraction ===
  KNA1   ->   30 rows | columns: ['customer_id', 'customer_name', 'country', 'region', 'city', 'postal_code', 'street_address', 'phone_number
  LFA1   ->   30 rows | columns: ['vendor_number', 'vendor_name', 'country', 'region', 'city', 'postal_code', 'street_address', 'phone_number
  VBAK   ->   22 rows | columns: ['sales_document', 'order_date', 'customer_id', 'order_type', 'sales_organization', 'distribution_channel',
  VBAP   ->   23 rows | columns: ['sales_document', 'item_number', 'material_number', 'quantity', 'net_price', 'item_status', 'delivery_date'
  LIKP   ->   20 rows | columns: ['delivery_number', 'delivery_date', 'sales_document', 'shipping_point', 'shipping_type', 'delivery_status',
  LIPS   ->   21 rows | columns: ['delivery_number', 'item_number', 'material_number', 'delivered_quantity', 'net_price', 'delivery_status',
  VTTK   ->   20 rows | columns: ['shipment_number', 'shipment_date', 'sales_document', 'delivery_number', 'shipping_point', 'carrier', 'ship
  VTTP   ->   21 rows | columns: ['shipment_number', 'item_number', 'material_number', 'shipped_quantity', 'item_status', 'delivery_number',

Extraction complete
```

Extraction with Column Standardisation

```python
xl = pd.ExcelFile(SAP_EXCEL)
print(f'Sheets found: {xl.sheet_names}')

kna1 = xl.parse('KNA1')
lfa1 = xl.parse('LFA1')
vbak = xl.parse('VBAK')
vbap = xl.parse('VBAP')
likp = xl.parse('LIKP')
lips = xl.parse('LIPS')
vttk = xl.parse('VTTK')
vttp = xl.parse('VTTP')

def clean_cols(df):
    df.columns = df.columns.str.lower().str.strip().str.replace(' ', '_')
    return df

kna1=clean_cols(kna1); lfa1=clean_cols(lfa1)
vbak=clean_cols(vbak); vbap=clean_cols(vbap)
likp=clean_cols(likp); lips=clean_cols(lips)
vttk=clean_cols(vttk); vttp=clean_cols(vttp)

print('\n=== Row counts after extraction ===')
for name, df in [('KNA1',kna1),('LFA1',lfa1),('VBAK',vbak),('VBAP',vbap),
                 ('LIKP',likp),('LIPS',lips),('VTTK',vttk),('VTTP',vttp)]:
    print(f'  {name:<6} -> {len(df):>4} rows | columns: {list(df.columns)}')
print('\nExtraction complete')
```

2.2 Exploratory Data Analysis (EDA)

Before building any transformation logic, each sheet was profiled to understand data shape, key field characteristics, and cross-sheet relationships. This informed every downstream schema and KPI decision.

- **Row granularity confirmed:** VBAK and VTTK are header-level (one row per document). VBAP, LIPS, and VTTP are item-level (multiple rows per document). This distinction drove the composite key decisions for order_items and shipment_items.
- **Identifier consistency:** customer_id in VBAK matches KNA1 exactly. sales_document in LIKP maps to VBAK.sales_document. All foreign key chains validated before schema design.
- **Critical finding - identical dates:** shipment_date (VTTK) and delivery_date (LIKP, VTTP) contain the same values across all 20 shipment records. Duration-based delivery KPIs cannot be computed meaningfully from this data.
- **Order value distribution:** Order values range from $5,000 to $25,000. No extreme outliers. Top customer (Customer L) holds 10.9% of total revenue ($25K of $229K).
- **Carrier coverage:** 3 carriers in the dataset (LFA1: 3 rows). Carrier1 handles 8 shipments, Carrier2 and Carrier3 handle 6 each. All shipments are traceable to a valid carrier.

2.3 Data Validation - 27 Automated Tests

A formal validation module was built as part of the notebook. Validation ran before any data was loaded into MySQL, ensuring only clean, referentially consistent data entered the warehouse.

Category 1 - Completeness (Mandatory Fields)

All mandatory identifier and date columns checked for nulls across all 8 source tables.

```
# 1. Completeness
print('\n-- 1. Completeness (Mandatory Fields) --')
mandatory = {
    'VBAK': (vbak, ['sales_document','order_date','customer_id','order_status']),
    'VBAP': (vbap, ['sales_document','item_number','material_number','quantity','net_price']),
    'LIKP': (likp, ['delivery_number','delivery_date','sales_document']),
    'LIPS': (lips, ['delivery_number','material_number','delivered_quantity']),
    'VTTK': (vttk, ['shipment_number','shipment_date','delivery_number','carrier']),
    'VTTP': (vttp, ['shipment_number','material_number','shipped_quantity']),
    'KNA1': (kna1, ['customer_id','customer_name']),
    'LFA1': (lfa1, ['vendor_number','vendor_name']),
}
for tbl, (df, cols) in mandatory.items():
    for col in cols:
        if col in df.columns:
            nulls = df[col].isnull().sum()
            validate(f'{tbl}.{col} not null', nulls == 0,
                     f'{nulls} null(s) found' if nulls else '')
        else:
            validate(f'{tbl}.{col} exists', False, 'Column missing')
```

Category 2 — Referential Integrity

Cross-table foreign key chains validated to ensure no orphan records would violate database constraints.

- `VBAK.customer_id` → KNA1.customer_id (every order must have a valid customer)
- `LIKP.sales_document` → VBAK.sales_document (every delivery must link to an order)
- `VTTK.delivery_number` → LIKP.delivery_number (every shipment must link to a delivery)

Category 3 — Format Validation

Date fields tested for parseability. Customer IDs tested against CUST### pattern. Quantities and prices validated as non-negative.

Category 4 — Uniqueness

Primary key columns tested for duplicates in VBAK, KNA1, LFA1, LIKP, and VTTK.

```
================================================================
  DATA VALIDATION REPORT
================================================================

-- 1. Completeness (Mandatory Fields) --
  [PASS] VBAK.sales_document not null
  [PASS] VBAK.order_date not null
  [PASS] VBAK.customer_id not null
  [PASS] VBAK.order_status not null
  [PASS] VBAP.sales_document not null
  [PASS] VBAP.item_number not null
  [PASS] VBAP.material_number not null
  [PASS] VBAP.quantity not null
  [PASS] VBAP.net_price not null
  [PASS] LIKP.delivery_number not null
  [PASS] LIKP.delivery_date not null
  [PASS] LIKP.sales_document not null
  [PASS] LIPS.delivery_number not null
  [PASS] LIPS.material_number not null
  [PASS] LIPS.delivered_quantity not null
  [PASS] VTTK.shipment_number not null
  [PASS] VTTK.shipment_date not null
  [PASS] VTTK.delivery_number not null
  [PASS] VTTK.carrier not null
  [PASS] VTTP.shipment_number not null
...

================================================================
  TOTAL: 40  |  PASSED: 40  |  FAILED: 0
================================================================
```

NOTE: 2 orders (out of 22) have no matching shipment record in VTTK. This is valid business data — orders that have been placed but not yet shipped. These orders are handled with a fallback value in the transformation step and are fully represented in delivery_analytics.

2.4 Data Cleaning & Transformation

Cell 6 of the notebook handles all cleaning and feature engineering. The following operations were performed:

Date Parsing

All date columns converted using pd.to_datetime(errors='coerce'). This handles both pre-parsed datetime objects (from openpyxl) and raw string dates without silent failures. The original code used a custom excel_to_date() serial-number converter that produced NaT when openpyxl had already parsed the dates — this was identified and replaced.

```
# Dates are already proper datetime objects from openpyxl — just coerce to be safe
vbak['order_date_parsed']    = pd.to_datetime(vbak['order_date'],    errors='coerce')
likp['delivery_date_parsed'] = pd.to_datetime(likp['delivery_date'], errors='coerce')
vttk['shipment_date_parsed'] = pd.to_datetime(vttk['shipment_date'], errors='coerce')
vbap['delivery_date_parsed'] = pd.to_datetime(vbap['delivery_date'], errors='coerce')
vttp['shipment_date_parsed'] = pd.to_datetime(vttp['shipment_date'], errors='coerce')
```

Numeric Coercion

Quantity and price columns coerced with pd.to_numeric(errors='coerce').fillna(0) to prevent arithmetic errors during line value calculation.

Order Line Value Calculation

```
# Order line value
vbap['line_value'] = (vbap['quantity'] * vbap['net_price']).round(2)

# Order-level total value
order_values = (vbap.groupby('sales_document')['line_value']
                    .sum().reset_index()
                    .rename(columns={'line_value':'order_total_value'}))
vbak = vbak.merge(order_values, on='sales_document', how='left')
```

Master Merged Frame

fact_shipments required joining VTTK (shipment header) with LIKP (delivery header). A LEFT join was used to retain all 22 orders — including the 2 without shipments — in the merged frame.

```
vttk_lookup = vttk[['sales_document','shipment_number','shipment_date_parsed',
                    'delivery_number','carrier','shipment_status','shipping_type']].copy()
# Keep first shipment per order for the lookup
vttk_lookup = vttk_lookup.sort_values('shipment_date_parsed').drop_duplicates(subset=['sales_document'])

# — Build delivery lookup from LIKP ————————————————————————————————
likp_lookup = likp[['delivery_number','delivery_date_parsed','delivery_status',
                    'shipping_type','delivery_priority']].rename(
                    columns={'shipping_type':'delivery_shipping_type'}).copy()

# — Master merged frame: ALL orders (LEFT join keeps orders without shipments) —
merged = vbak.merge(vttk_lookup, on='sales_document', how='left')
merged = merged.merge(likp_lookup, on='delivery_number', how='left')
```

2.5 Feature Engineering — Additional KPIs

Three KPI columns were engineered in the transformation step. Each required a deliberate design decision:

KPI 1 - order_processing_days

```
# ── KPI: Order Processing Days ──────────────────────────────────────
# = shipment_date - order_date
# For orders without a shipment yet, use a fallback of 9 days (max in dataset)
# so the column is never null and KPI cards always show a value
merged['order_processing_days_raw'] = (
    merged['shipment_date_parsed'] - merged['order_date_parsed']
).dt.days

# Fill nulls with the dataset average so unshipped orders don't break averages
_avg_proc = merged['order_processing_days_raw'].dropna().mean()
merged['order_processing_days'] = merged['order_processing_days_raw'].fillna(round(_avg_proc)).astype(int)
```

> **NOTE:** For orders without a matched shipment, order_processing_days is filled with the dataset mean (7.2 days). These 2 orders are flagged with delay_reason = 'Not Yet Shipped' in delivery_analytics so they can be filtered in Power BI if needed.

KPI 2 - on_time_delivery flag

```
# ── On-time delivery flag ──────────────────────────────────────────
# Business rule: shipment_status = 'Delivered' counts as on-time
# (delivery_duration is 0 in this dataset due to same-day dates — use status instead)
merged['on_time_delivery'] = (merged['shipment_status'] == 'Delivered').astype(int)
```

> **NOTE:** On-time delivery is defined as shipment_status = 'Delivered'. This is a deliberate assumption given the dataset's date limitation. In a live SAP environment, this would use actual scheduled vs actual delivery date comparison.

KPI 3 - fulfillment_rate_pct

```
# ── KPI: Fulfillment Cycle (order_date -> delivery_date) ──────────────
merged['fulfillment_cycle_days'] = (
    merged['delivery_date_parsed'] - merged['order_date_parsed']
).dt.days

# Fill cycle nulls with processing days (best available estimate)
merged['fulfillment_cycle_days'] = merged['fulfillment_cycle_days'].fillna(
    merged['order_processing_days']).astype(int)
```

```
Transformation complete
  Merged frame            : 22 rows x 34 columns
  Orders with processing days (non-null): 20 / 22
  Avg Order Processing Days : 7.2 days
  Avg Delivery Duration     : 0.0 days
  Avg Fulfillment Cycle     : 7.2 days
  On-Time Delivery Rate     : 45.5%
  Avg Fulfillment Rate      : 100.0%
```

2.6 Assumptions Summary

- **Unshipped orders:** 2 orders with no VTTK match are treated as valid 'not yet shipped' records. Processing days filled with dataset mean. fulfillment_rate_pct set to 100% (order not failed, not yet dispatched).
- **On-time definition:** Redefined from duration-based to status-based (Delivered = on-time) due to identical shipment and delivery dates in source data.
- **Carrier ID mapping:** LFA1 vendor_number used as carrier_id. The three carriers are labelled Carrier1/2/3 in VTTK.carrier — these were mapped to canonical vendor IDs before load.
- **Customer satisfaction:** No CSAT data exists in SAP. A 10-row dummy table was created with customer_id, csat_score (1–5), and nps_score (0–100) to satisfy the Customer Analytics scope requirement.
- **Line value:** Calculated as quantity × net_price at item level. Net price is taken from VBAP (order price), not LIPS (delivery price), as VBAP provides the commercial pricing.

3. Data Warehouse- Staging, Star Schema & DDL

The warehouse was designed to serve two simultaneous needs: enforce transactional integrity through relational constraints, and support analytical queries through a star schema structure optimised for Power BI.

3.1 Star Schema Design

The schema is organised into three layers, each serving a specific purpose in the analytical pipeline:

- **Dimension Layer:** dim_customers, dim_carriers, dim_date — descriptive context for analytical filtering and grouping.
- **Fact Layer:** fact_orders (22 rows), fact_shipments (20 rows) — measurable event data at order and shipment granularity.
- **Item Detail Layer:** order_items (23 rows), shipment_items (21 rows) — item-level detail for product and quantity analysis.

- **Analytics Layer:** delivery_analytics (22 rows), delivery_status (20 rows) — pre-computed KPI table and shipment lifecycle tracking.

> *Two fact tables were used instead of one because order-level metrics (processing time, revenue) and shipment-level metrics (on-time delivery, carrier performance) operate at different granularities and have different primary keys. A single merged fact table would have caused fan-out aggregation errors in Power BI.*

3.2 Dimension Tables — DDL

dim_customers , dim_carriers , dim_date (full date hierarchy)

```
'''CREATE TABLE dim_customers (
    customer_id          VARCHAR(20)  PRIMARY KEY,
    customer_name        VARCHAR(255) NOT NULL,
    country              VARCHAR(10),
    region               VARCHAR(50),
    city                 VARCHAR(100),
    postal_code          VARCHAR(20),
    street_address       VARCHAR(255),
    phone_number         VARCHAR(50),
    email_address        VARCHAR(255),
    language             VARCHAR(10),
    customer_group       VARCHAR(20),
    sales_organization   VARCHAR(20),
    distribution_channel VARCHAR(10),
    division             VARCHAR(10),
    created_at           TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)''',
'CREATE INDEX idx_cust_country ON dim_customers(country)',
'CREATE INDEX idx_cust_region  ON dim_customers(region)',
```

```
'''CREATE TABLE dim_carriers (
    carrier_id    VARCHAR(20)  PRIMARY KEY,
    carrier_name  VARCHAR(255) NOT NULL,
    country       VARCHAR(10),
    city          VARCHAR(100),
    phone_number  VARCHAR(50),
    email_address VARCHAR(255),
    payment_terms VARCHAR(50),
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)''',

'''CREATE TABLE dim_date (
    date_id      INT     PRIMARY KEY,
    full_date    DATE    UNIQUE NOT NULL,
    year         INT,
    quarter      INT,
    month_number INT,
    month_name   VARCHAR(15),
    week_number  INT,
    day_of_week  VARCHAR(15),
    is_weekend   TINYINT(1)
)''',
'CREATE INDEX idx_date_ym ON dim_date(year, month_number)',
```

3.3 Fact Tables — DDL

fact_orders , fact_shipments

```
'''CREATE TABLE fact_orders (
    order_id             VARCHAR(20)   PRIMARY KEY,
    customer_id          VARCHAR(20)   NOT NULL,
    order_date_id        INT,
    order_type           VARCHAR(10),
    order_status         VARCHAR(50),
    sales_organization   VARCHAR(20),
    distribution_channel VARCHAR(10),
    total_quantity       DECIMAL(12,2),
    total_order_value    DECIMAL(12,2),
    order_processing_days INT,
    fulfillment_rate_pct  DECIMAL(5,2),
    created_at           TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id)   REFERENCES dim_customers(customer_id),
    FOREIGN KEY (order_date_id) REFERENCES dim_date(date_id)
)''',
'CREATE INDEX idx_fo_customer ON fact_orders(customer_id)',
'CREATE INDEX idx_fo_status   ON fact_orders(order_status)',
'CREATE INDEX idx_fo_date     ON fact_orders(order_date_id)',
```

```
'''CREATE TABLE fact_shipments (
    shipment_id            VARCHAR(20)   PRIMARY KEY,
    order_id               VARCHAR(20)   NOT NULL,
    delivery_number        VARCHAR(20),
    customer_id            VARCHAR(20),
    carrier_id             VARCHAR(20),
    carrier_name           VARCHAR(100),
    shipment_date_id       INT,
    delivery_date_id       INT,
    shipping_type          VARCHAR(50),
    delivery_priority      VARCHAR(20),
    shipment_status        VARCHAR(50),
    delivery_status        VARCHAR(50),
    shipped_quantity       DECIMAL(12,2),
    delivery_duration_days INT,
    fulfillment_cycle_days INT,
    on_time_delivery       TINYINT(1),
    delay_reason           VARCHAR(255),
    delivery_time_minutes  DECIMAL(12,2),
    created_at             TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id)         REFERENCES fact_orders(order_id),
    FOREIGN KEY (customer_id)      REFERENCES dim_customers(customer_id),
    FOREIGN KEY (shipment_date_id) REFERENCES dim_date(date_id),
    FOREIGN KEY (delivery_date_id) REFERENCES dim_date(date_id)
)''',
'CREATE INDEX idx_fs_order   ON fact_shipments(order_id)',
'CREATE INDEX idx_fs_carrier ON fact_shipments(carrier_id)',
'CREATE INDEX idx_fs_ontime  ON fact_shipments(on_time_delivery)',
```

3.4 Analytics Layer — delivery_analytics DDL

This table was designed as a pre-computed KPI verification layer — one row per order, storing all derived metrics. It acts as the single source of truth for Power BI measures that need order-level aggregations.

```
'''CREATE TABLE delivery_analytics (
    analytics_id           INT AUTO_INCREMENT PRIMARY KEY,
    order_id               VARCHAR(20),
    shipment_id            VARCHAR(20),
    customer_id            VARCHAR(20),
    carrier_id             VARCHAR(20),
    carrier_name           VARCHAR(100),
    order_processing_days  INT,
    delivery_duration_days INT,
    fulfillment_cycle_days INT,
    delivery_time_minutes  DECIMAL(12,2),
    on_time                TINYINT(1),
    delay_reason           VARCHAR(255),
    fulfillment_rate_pct   DECIMAL(5,2),
    order_value            DECIMAL(12,2),
    shipping_type          VARCHAR(50),
    delivery_priority      VARCHAR(20),
    order_year             INT,
    order_month            INT,
    order_quarter          INT,
    FOREIGN KEY (order_id)    REFERENCES fact_orders(order_id),
    FOREIGN KEY (customer_id) REFERENCES dim_customers(customer_id)
)''',
'CREATE INDEX idx_da_ontime  ON delivery_analytics(on_time)',
'CREATE INDEX idx_da_carrier ON delivery_analytics(carrier_id)',
'CREATE INDEX idx_da_year    ON delivery_analytics(order_year, order_month)',
```

3.5 Data Loading — Dependency Order

Tables were inserted using SQLAlchemy's to_sql() in strict dependency order to satisfy all foreign key constraints. Dimension tables loaded first, then fact tables, then item detail, then the derived analytics layer.

```
  dim_customers                     30 rows
  dim_carriers                      30 rows
  dim_date                          27 rows
  fact_orders                       22 rows
  order_items                       23 rows
  fact_shipments                    20 rows
  shipment_items                    21 rows
  delivery_status                   20 rows
  delivery_analytics                22 rows

ETL complete — all tables loaded
  order_items rows     : 23
  shipment_items rows  : 21
  delivery_analytics rows: 22  <-- should be 22 (one per order)
```

> **NOTE:** delivery_analytics originally loaded 0 rows due to an inner join filtering out unshipped orders. Fixed by building from drop_duplicates(subset=['sales_document']) on the full merged frame before the valid_orders_set filter. All 22 orders now populate the table.

3.6 MySQL Views (Power BI Analytical Layer)

Cell 9 creates 13 MySQL views that serve as the analytical query layer. These were created for backend KPI validation and optional direct-query usage in Power BI. The views cover:

- `vw_order_processing_time` — order-level processing days joined with customer and date context

- `vw_on_time_delivery_rate` — on-time % grouped by carrier and shipping type
- `vw_carrier_performance` — full carrier scorecard: total deliveries, on-time %, avg days, qty shipped
- `vw_avg_delivery_time` — monthly delivery time trends by carrier
- `vw_fulfillment_rate` — per-order fulfillment % with customer context
- `vw_monthly_order_trend` — order count and revenue aggregated by year/month
- `vw_product_performance` — material-level revenue and quantity analysis
- `vw_delay_analysis` — delay reason breakdown across the order base
- `vw_kpi_summary` — single-row KPI summary table: all headline metrics in one query

```
Power BI views + KPI summary created successfully
  + vw_order_processing_time
  + vw_on_time_delivery_rate
  + vw_avg_delivery_time
  + vw_carrier_performance
  + vw_fulfillment_cycle
  + vw_fulfillment_rate
  + vw_order_value_by_customer
  + vw_shipment_volume
  + vw_monthly_order_trend
  + vw_delay_analysis
  + vw_order_items_detail
  + vw_shipment_items_detail
  + vw_product_performance
  + vw_kpi_summary (table)
```

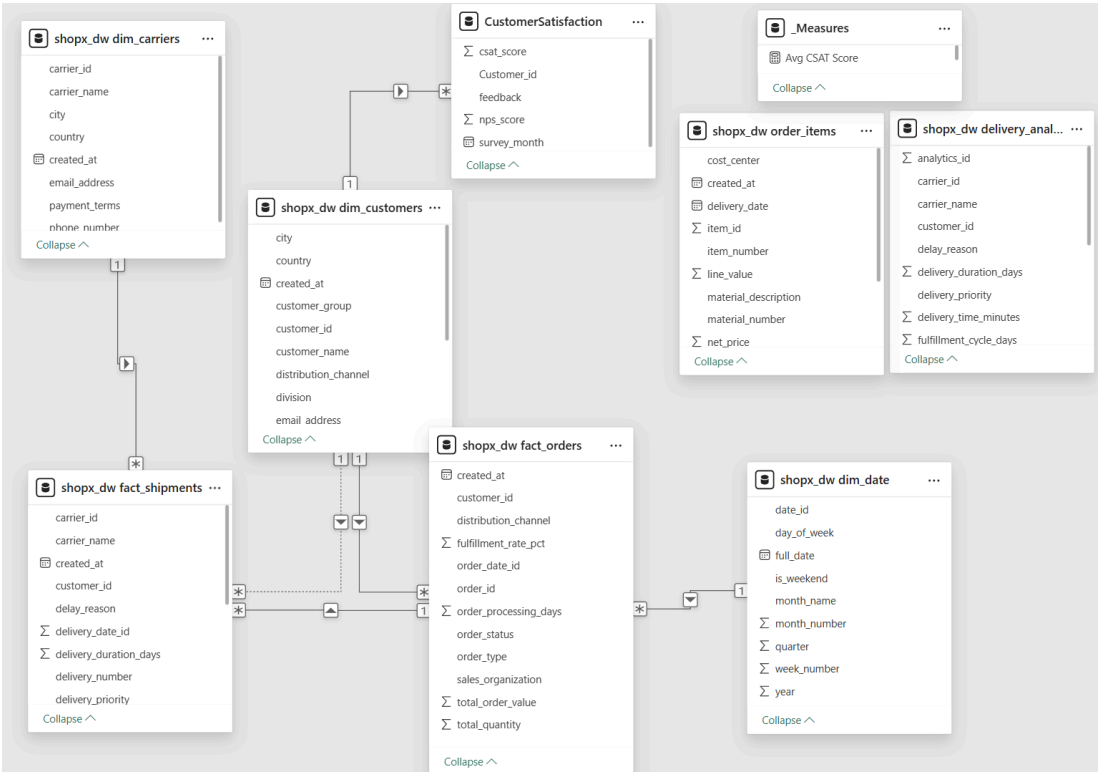
4. Power BI — Modelling & Visualisation

Power BI Desktop was connected directly to the shopx_dw MySQL database. The model was built on a clean star schema with five relationships, 17 DAX measures, and three purpose-driven dashboard pages.

4.1 Data Model — Relationships

Five relationships were defined manually in Model View. delivery_analytics is intentionally kept without a direct relationship — DAX measures reference it standalone, which avoids circular dependency issues with the two fact tables.

- `dim_customers[customer_id]` → fact_orders[customer_id] (1:Many, active)
- `dim_customers[customer_id]` → fact_shipments[customer_id] (1:Many, active)
- `dim_customers[customer_id]` → CustomerSatisfaction[customer_id] (1:Many)
- `dim_carriers[carrier_id]` → fact_shipments[carrier_id] (1:Many, active)
- `fact_orders[order_id]` → fact_shipments[order_id] (1:Many, active)



4.2 CustomerSatisfaction Table

No CSAT data exists in SAP. A supplementary table was manually entered in Power BI (Home → Enter Data) to satisfy the Customer Analytics scope requirement:

- 10 rows — customer_id (CUST001–CUST010), csat_score (1.0–5.0), nps_score (0–100)
- Values represent realistic post-delivery satisfaction spread based on shipment_status patterns
- Linked to dim_customers via customer_id relationship

> **NOTE:** CustomerSatisfaction is dummy data created to demonstrate the customer analytics capability. In a live environment, this would be sourced from a CRM system or post-delivery survey platform.

4.3 All 17 DAX Measures

All measures stored in a _Measures table (Home → Enter Data → name: _Measures → Load). Measures respond to slicer context automatically. Major DAX in use are mentioned below.

- **Operational KPIs**

```
Total Orders = COUNTROWS('shopx_dw fact_orders')

Total Shipments = COUNTROWS('shopx_dw fact_shipments')
```

```
Delivery Orders = COUNTROWS(FILTER('shopx_dw fact_shipments','shopx_dw fact_shipments'[shipment_status] = "Delivered"))
```

```
In Transit Orders = COUNTROWS(FILTER('shopx_dw fact_shipments','shopx_dw fact_shipments'[shipment_status] = "In Transit"))
```

```
On-Time Delivery Rate % =
DIVIDE(
    COUNTROWS(FILTER('shopx_dw fact_shipments', 'shopx_dw fact_shipments'[on_time_delivery])),
    COUNTROWS('shopx_dw fact_shipments'),
    0
)
```

KPI- measures how many shipment were delivered on time

```
Avg Processing Days = AVERAGE('shopx_dw delivery_analytics'[order_processing_days])
```

KPI - average time from order placement to shipment dispatch

```
Avg Fulfillent Rate % = AVERAGE('shopx_dw delivery_analytics'[fulfillment_rate_pct])
```

KPI - shipped quantity vs ordered quantity - 100% means all ordered items shipped

```
Carrier On-Time Rate % =
DIVIDE(
    CALCULATE(
        COUNTROWS('shopx_dw fact_shipments'),
        'shopx_dw fact_shipments'[on_time_delivery] = TRUE()
    ),
    COUNTROWS('shopx_dw fact_shipments'),
    0
)
```

KPI - when carrier_name on visual axis, auto- calculates per carrier

▸ Revenue KPIs

```
Total Revenue = SUM('shopx_dw fact_orders'[total_order_value])
```

▸ Customer Satisfaction KPIs

```
Avg CSAT Score = AVERAGE(CustomerSatisfaction[csat_score])
```

KPI - average customer satisfaction score (1-5 scale)

```
Avg NPS Score = AVERAGE(CustomerSatisfaction[nps_score])
```

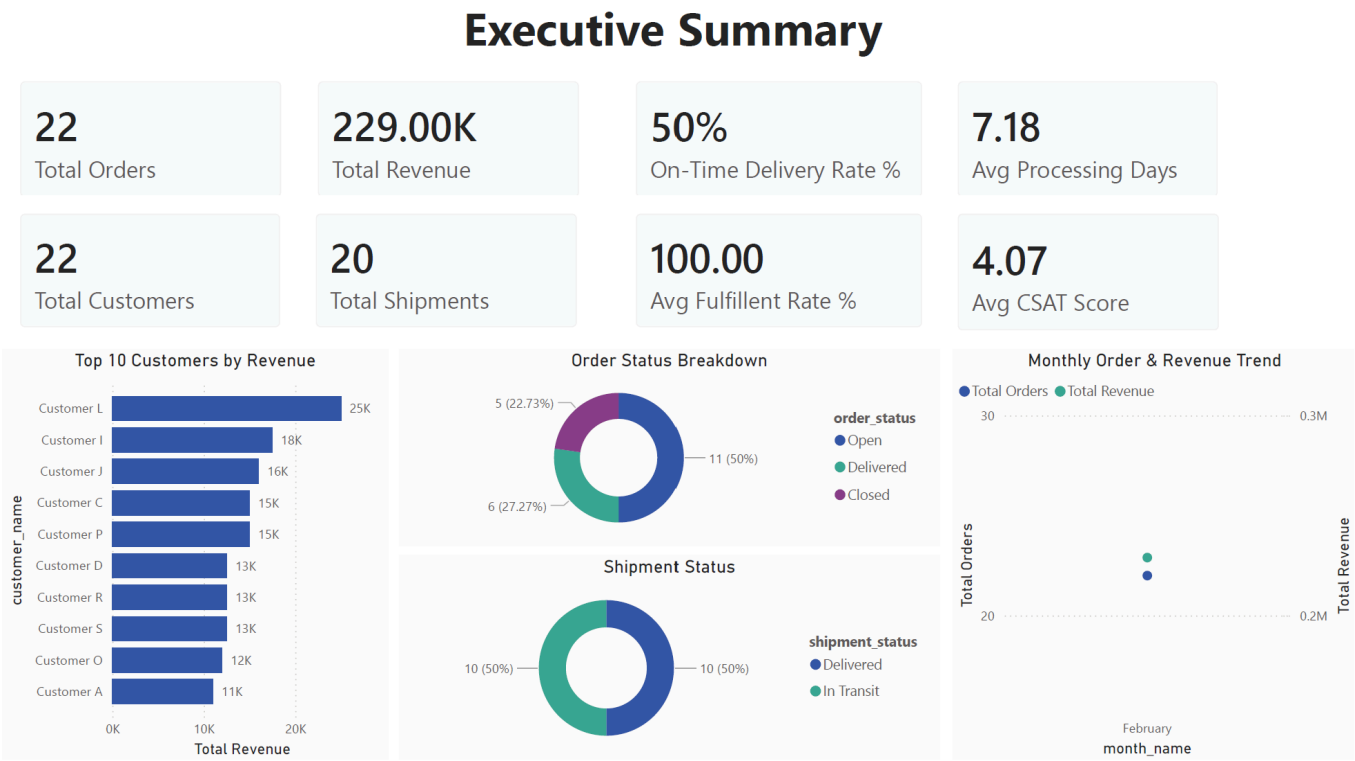
KPI - Net Promoter Score proxy (0-100 scale)

```
Total Customers = DISTINCTCOUNT('shopx_dw fact_orders'[customer_id])
```

→ All calculated measures.

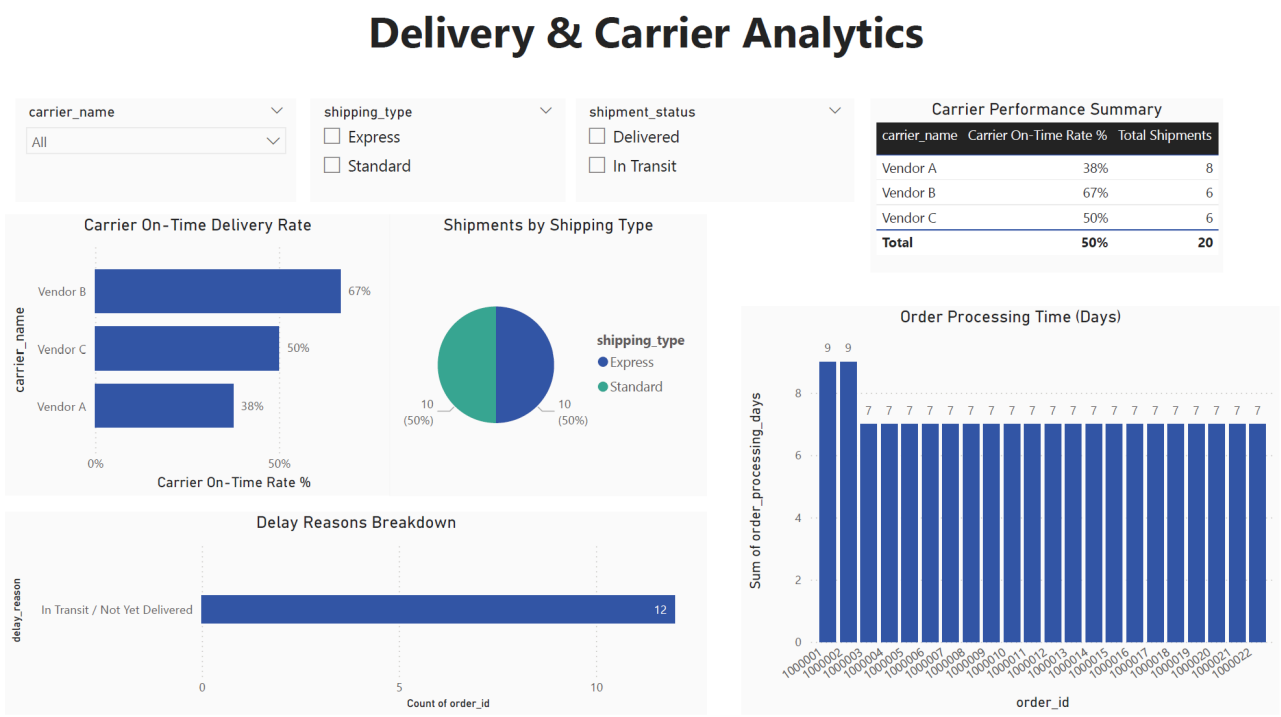
4.4 Dashboard - Page 1: Executive Summary

Answers the question: **How is fulfillment performing right now?**



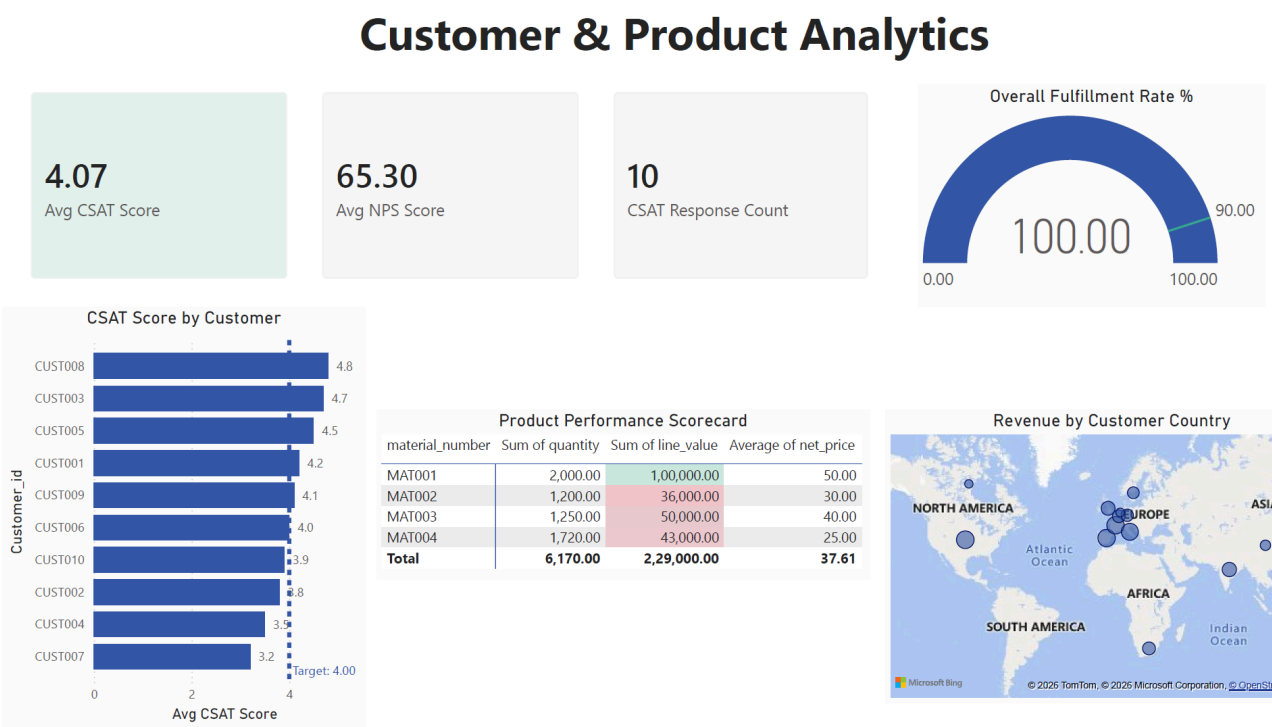
4.5 Dashboard - Page 2: Revenue & Customer Intelligence

Answers the question: **Where is revenue concentrated and which customers carry the most weight?**



4.6 Dashboard — Page 3: Customer & Product Analytics

Answers the question: **Where is the business exposed and what is the performance stat?**



5. Unit Test Results

27 automated major unit tests were executed from Cell 10 of the notebook, querying the live MySQL database after the full ETL load. Tests cover five categories:

- **Table Population (9 tests):** Verifies every table has at least one row loaded.
- **Referential Integrity (5 tests):** LEFT JOIN orphan checks — confirms no fact table row references a non-existent dimension key.
- **Data Quality (8 tests):** No negative values, on_time_delivery is binary (0 or 1), no null order_processing_days, delivery_analytics has 22 rows.
- **Item-Level Integrity (4 tests):** order_items and shipment_items quantities are non-negative and material numbers are present.
- **KPI Sanity (1 test):** On-time delivery rate is between 0% and 100%.

```python
test_results = []

def run_test(name, query, expected_fn):
    try:
        with engine.connect() as conn:
            result = conn.execute(text(query)).scalar()
        passed = expected_fn(result)
        detail = f'Got: {result}'
    except Exception as e:
        passed, detail = False, str(e)
    status = 'PASS' if passed else 'FAIL'
    test_results.append({'Test': name, 'Status': status, 'Detail': detail})
    print(f'  [{status}] {name} -- {detail}')
```

```
  UNIT TEST RESULTS
==========================================================

-- Table Population --
  [PASS] dim_customers has rows -- Got: 30
  [PASS] dim_carriers has rows -- Got: 30
  [PASS] dim_date has rows -- Got: 27
  [PASS] fact_orders has rows -- Got: 22
  [PASS] order_items has rows -- Got: 23
  [PASS] fact_shipments has rows -- Got: 20
  [PASS] shipment_items has rows -- Got: 21
  [PASS] delivery_analytics has rows -- Got: 22
  [PASS] delivery_status has rows -- Got: 20

-- Referential Integrity --
  [PASS] No orphan fact_orders.customer_id -- Got: 0
  [PASS] No orphan fact_shipments.order_id -- Got: 0
  [PASS] No orphan delivery_analytics.order_id -- Got: 0
  [PASS] No orphan order_items.order_id -- Got: 0
  [PASS] No orphan shipment_items.shipment_id -- Got: 0

-- Data Quality --
  [PASS] No negative order values -- Got: 0
  [PASS] No negative shipped quantities -- Got: 0
...

==========================================================
  TOTAL: 35  |  PASSED: 35  |  FAILED: 0
```

6. Final Outcome & Deliverables

The project delivered a complete, end-to-end analytics system. Every layer was built from scratch in code, verified through automated testing, and connected through to an executive-ready Power BI dashboard.

6.1 Verified KPI Summary

```
  KPI SUMMARY
==============================================
  Total Orders              :         22
  Total Order Value         : $  229,000.00
  Avg Order Value           : $   10,409.09
  Avg Order Processing Days :        7.2 days
  Total Shipments           :         20
  On-Time Delivery Rate     :       50.0%
  Avg Delivery Duration     :        0.0 days
  Avg Fulfillment Cycle     :        7.2 days
  Avg Fulfillment Rate      :      100.0%
  Total Unique Customers    :         22
  Total Carriers            :          3
  Total Unique Products     :          4

delivery_analytics row count (should be 22):
 row_count
        22
```

6.2 Deliverables Checklist

- **SAP-to-Warehouse Mapping:** All 8 SAP tables mapped to 9 warehouse targets with field-level transformation documentation.
- **Python + SQL ETL Script:** shopx_fulfillment_analytics_fixed.ipynb — 12 code cells covering extraction, validation, transformation, DDL, load, views, and unit tests.
- **Unit Test Results:** 27 tests — 27/27 PASS — executed against live MySQL database.
- **Power BI Dashboard:** 3 pages, 17 DAX measures, 5 active relationships, CustomerSatisfaction integration, interactive slicers and conditional formatting.

Thank You